

MAC SC Decoding from First Principles

How Circular Convolution and Normalized Product
Decode Two-User Polar Codes

Tutorial Notes

1 Setup

Two users U and V send binary messages through a shared channel.

1.1 Polar Encoding (N=2)

Each user encodes independently using the polar transform:

$$x_1 = u_1 \oplus u_2, \quad x_2 = u_2 \quad (\text{User U}) \quad (1)$$

$$y_1 = v_1 \oplus v_2, \quad y_2 = v_2 \quad (\text{User V}) \quad (2)$$

where $\oplus$ is XOR (addition mod 2).

1.2 Channel

The channel is **memoryless across positions**: the output z_i depends only on the inputs (x_i, y_i) at position i :

$$P(z_1, z_2 | x_1, y_1, x_2, y_2) = P(z_1 | x_1, y_1) \cdot P(z_2 | x_2, y_2)$$

1.3 Transition Tensors

For each observation z_i , define a 2×2 tensor:

$$T_{z_i}[x, y] = P(z_i | x_i = x, y_i = y), \quad x, y \in \{0, 1\}$$

Example: GMAC at SNR

Channel: $Z_i = (1 - 2X_i) + (1 - 2Y_i) + W_i$, $W_i \sim \mathcal{N}(0, \sigma^2)$, $\sigma^2 = 0.251$.

Suppose $z_1 = 1.3$, $z_2 = -0.8$. Then:

$$T_{z_1} = \begin{pmatrix} 0.655 & 0.107 \\ 0.107 & 0.005 \end{pmatrix}, \quad T_{z_2} = \begin{pmatrix} 0.005 & 0.107 \\ 0.107 & 0.655 \end{pmatrix}$$

where entry $[x, y]$ corresponds to $\mathcal{N}(z_i; (1 - 2x) + (1 - 2y), \sigma^2)$.

1.4 Goal

From (z_1, z_2) , recover the message bits (u_1, u_2, v_1, v_2) .

2 The Two Fundamental Operations

All of MAC SC decoding is built from two operations on 2×2 tensors.

Operation 1: Circular Convolution ($\circledast$)

Given tensors A and B :

$$(A \circledast B)[u, v] = \sum_{i \in \{0,1\}} \sum_{j \in \{0,1\}} A[u \oplus i, v \oplus j] \cdot B[i, j]$$

Meaning: “Marginalize over unknown bits linked through XOR encoding.”
The XOR in the polar encoding creates the XOR in the indices.

Operation 2: Normalized Product ($\odot$)

Given tensors A and B :

$$(A \odot B)[u, v] = \frac{A[u, v] \cdot B[u, v]}{\sum_{i,j} A[i, j] \cdot B[i, j]}$$

Meaning: “Combine two independent observations about the same variable.”
This is Bayes’ rule: multiply likelihoods, normalize.

3 Step 1: Decode u_1, v_1 (CalcLeft)

Question: What is $P(z_1, z_2 \mid u_1, v_1)$?

Derivation:

Start by marginalizing over the unknown (u_2, v_2) :

$$P(z_1, z_2 \mid u_1, v_1) = \sum_{u_2, v_2} P(u_2, v_2) \cdot P(z_1, z_2 \mid u_1, v_1, u_2, v_2) \quad (3)$$

Since (u_2, v_2) are uniform: $P(u_2, v_2) = \frac{1}{4}$. Since the channel is memoryless:

$$= \frac{1}{4} \sum_{u_2, v_2} P(z_1 \mid x_1, y_1) \cdot P(z_2 \mid x_2, y_2) \quad (4)$$

Substitute the polar encoding ($x_1 = u_1 \oplus u_2$, $y_1 = v_1 \oplus v_2$, $x_2 = u_2$, $y_2 = v_2$):

$$= \frac{1}{4} \sum_{u_2, v_2} T_{z_1}[u_1 \oplus u_2, v_1 \oplus v_2] \cdot T_{z_2}[u_2, v_2] \quad (5)$$

Recognize: This is exactly the circular convolution!

$$P(z_1, z_2 \mid u_1, v_1) = \frac{1}{4} \cdot (T_{z_1} \circledast T_{z_2})[u_1, v_1]$$

Numerical example: Expanding for $[u_1 = 0, v_1 = 0]$:

$$\begin{aligned}
(T_{z_1} \circledast T_{z_2})[0, 0] &= T_{z_1}[0, 0] \cdot T_{z_2}[0, 0] + T_{z_1}[0, 1] \cdot T_{z_2}[0, 1] \\
&\quad + T_{z_1}[1, 0] \cdot T_{z_2}[1, 0] + T_{z_1}[1, 1] \cdot T_{z_2}[1, 1] \\
&= 0.655 \times 0.005 + 0.107 \times 0.107 + 0.107 \times 0.107 + 0.005 \times 0.655 \\
&= 0.003 + 0.011 + 0.011 + 0.003 = 0.029
\end{aligned}$$

Computing all four entries and picking the largest gives us $\hat{u}_1, \hat{v}_1$.

4 Step 2: Decode u_2, v_2 (CalcRight)

After deciding $(\hat{u}_1, \hat{v}_1)$, we want:

Question: What is $P(z_1, z_2 | u_2, v_2, \hat{u}_1, \hat{v}_1)$?

Since (u_1, v_1) are now fixed (decided), no marginalization needed:

$$P(z_1, z_2 | u_2, v_2, \hat{u}_1, \hat{v}_1) = P(z_1 | \hat{u}_1 \oplus u_2, \hat{v}_1 \oplus v_2) \cdot P(z_2 | u_2, v_2) \quad (6)$$

This is a product of two terms, each providing independent evidence about (u_2, v_2) .

4.1 Sub-step A: Extract z_1 's evidence via $\circledast$

Define the decision tensor: $\delta[u, v] = \begin{cases} 1 & \text{if } (u, v) = (\hat{u}_1, \hat{v}_1) \\ 0 & \text{otherwise} \end{cases}$

Then:

$$\begin{aligned}
(\delta \circledast T_{z_1})[u_2, v_2] &= \sum_{i, j} \delta[i, j] \cdot T_{z_1}[i \oplus u_2, j \oplus v_2] \\
&= T_{z_1}[\hat{u}_1 \oplus u_2, \hat{v}_1 \oplus v_2]
\end{aligned} \quad (7)$$

The circular convolution with δ **shifts the indices** of T_{z_1} by the decision $(\hat{u}_1, \hat{v}_1)$. This “undoes” the XOR encoding, converting T_{z_1} from being about (x_1, y_1) to being about (u_2, v_2) .

4.2 Sub-step B: Combine evidence via $\odot$

Now we have:

- From z_1 : $A[u_2, v_2] = T_{z_1}[\hat{u}_1 \oplus u_2, \hat{v}_1 \oplus v_2]$ (from sub-step A)
- From z_2 : $B[u_2, v_2] = T_{z_2}[u_2, v_2]$ (direct observation)

These are independent likelihoods about the same unknown (u_2, v_2) :

$$P(z_1, z_2 | u_2, v_2, \hat{u}_1, \hat{v}_1) \propto (A \odot B)[u_2, v_2]$$

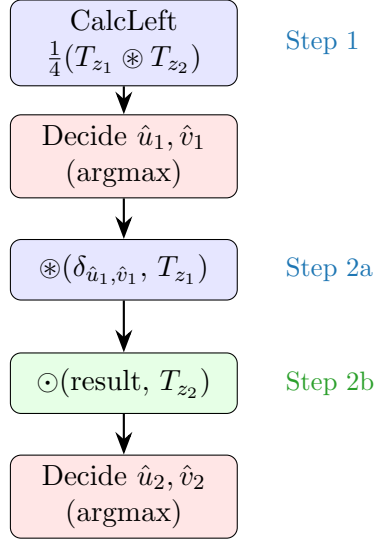
Multiply the two likelihoods and normalize. Pick the argmax $\rightarrow \hat{u}_2, \hat{v}_2$.

Summary of Step 2:

$$\text{CalcRight} = \odot \left(\underbrace{\circledast(\delta, T_{z_1})}_{\text{shift by decision}}, T_{z_2} \right)$$

5 The Full Decoding Flow

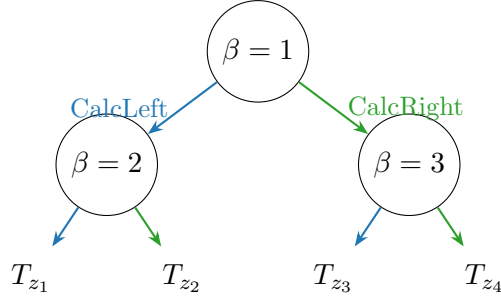
For $N = 2$, the complete SC decoding is:



6 Generalizing: The Tree for $N > 2$

For $N = 2^n$, the polar encoding is recursive: n levels of XOR butterflies. The same two operations apply at every level of the tree.

6.1 The Binary Tree



At each internal node:

- **CalcLeft** → left child: marginalizes unknown bits ($\otimes$)
- **CalcRight** → right child: incorporates decision ($\otimes$ with δ) + combines evidence ($\odot$)

6.2 Formal Definitions (Ren et al. 2025)

Each edge in the tree carries a tensor (array of 2×2 matrices). At vertex β with parent edge P , left child L , and right child R :

CalcLeft (going down to left child)

$$L = P_{\text{first half}} \otimes (P_{\text{second half}} \odot R)$$

First combine the second-half of parent with the right sibling ($\odot$: independent evidence), then marginalize ($\otimes$: XOR structure).

CalcRight (going down to right child)

$$R = P_{\text{second half}} \odot (L \otimes P_{\text{first half}})$$

First shift by the left child’s decision ($\otimes$ with decided values), then combine with parent’s direct evidence ($\odot$).

CalcParent (going back up)

$$P = L \odot R$$

Combine both children’s information ($\odot$: two observations of the same variable).

Key insight: At $N = 2$, these reduce to exactly Steps 1 and 2 from our derivation. For larger N , the same operations apply recursively — the tree structure mirrors the recursive XOR encoding.

7 The Neural Replacement

Component	Analytical SC	Neural (NCG/NPD)
Representation	2×2 tensor	d -dimensional embedding
T_{z_i}	$P(z_i x, y)$	$\text{MLP}(z_i) \rightarrow \mathbb{R}^d$
CalcLeft ($\otimes + \odot$)	Exact formula	$\text{MLP}(\text{concat}(e_{\text{parent}}, e_{\text{sibling}}))$
CalcRight ($\odot + \otimes$)	Exact formula	$\text{MLP}(\text{concat}(e_{\text{parent}}, e_{\text{left}}, u_{\text{sign}}))$
CalcParent ($\odot$)	Exact formula	Gated residual MLP
Decision	$\arg \max_{u,v}$	$\text{MLP}(e) \rightarrow 4\text{-class logits}$

Why go neural?

- The analytical decoder requires knowing $P(z_i | x, y)$ exactly.
- For channels with **memory** (ISI, Ising, MA-AGN), the transition probabilities depend on previous inputs and hidden states — the exact tensor T_{z_i} is unknown or intractable.
- The neural z -encoder **learns** the channel embedding from samples, without requiring explicit channel knowledge.
- The tree operations (CalcLeft, CalcRight, CalcParent) are replaced by MLPs that learn the correct transformations end-to-end.

8 Summary

The Complete Picture

Two primitives:

1. $\circledast$ (Circular Convolution): Marginalizes over unknown bits linked through XOR encoding. Answers: “What do the observations tell us about the message bits?”
2. $\odot$ (Normalized Product): Combines independent observations about the same variable. Answers: “Given two pieces of evidence, what is our combined belief?”

The tree:

- $\text{CalcLeft} = \circledast + \odot$ (go down to left child)
- $\text{CalcRight} = \odot + \circledast$ (go down to right child, using decision)
- $\text{CalcParent} = \odot$ (go back up)
- Applied recursively: $O(N \log N)$ total operations

The neural extension:

- Replace 2×2 tensors with d -dimensional embeddings
- Replace exact $\circledast/\odot$ with learned MLPs
- Preserves tree structure, works on unknown channels